NOM : _____

Examen final Programmation C++

27/01/2026 - Durée : 1h30 - Documents papiers autorisés.

Attention : Vous veillerez à écrire du code le plus propre possible respectant les règles présentées en cours. Notamment, vous ferez attention à nommer explicitement vos variables, vos fonctions ou vos méthodes, à utiliser à bon escient les références, à utiliser correctement le mot clé « const » dès que nécessaire, à respecter les principes SOLID, …

Exercice 1 : QCM – Compréhension des concepts fondamentaux du C++ (5 points)

Répondez aux 10 questions en reportant vos réponses dans le tableau ci-dessous. Attention, les questions peuvent comporter plusieurs réponses possibles.

Pour chaque question, 0.5 point est attribué si l'ensemble des réponses cochées est entièrement correct. Si la réponse comporte une seule erreur (oubli d'une réponse juste ou ajout d'une réponse fausse), 0,25 point est attribué. Sinon, la question est notée 0.

	a	b	c	d
Q01				
Q02				
Q03				
Q04				
Q05				
Q06				
Q07				
Q08				
Q09				
Q10				

Q01 Quel est l'intérêt principal de la programmation orientée objet ?

a) Organiser le code autour d'entités cohérentes représentant des concepts du problème.
b) Remplacer les fonctions par des classes.
c) Écrire des programmes plus courts.
d) Faciliter l'évolution, la maintenance et la réutilisation du code.

Q02 Quel est l'objectif principal des principes SOLID en programmation orientée objet ?

a) Imposer une manière unique d'écrire du code orienté objet.
b) Aider à concevoir des classes et des architectures plus robustes et évolutives.
c) Réduire le nombre total de classes dans un programme.
d) Garantir l'absence de bugs dans un programme.

Q03 En C++, à quoi servent les fichiers .h et .cpp ?

a) À savoir comment une classe peut être utilisée sans avoir besoin de connaître son implémentation.
b) À rendre possible la compilation séparée des différentes parties du programme.
c) À éviter les dépendances entre classes.
d) À permettre d'inclure des fichiers .h et .cpp dans d'autres fichiers selon les besoins.

Q04 Lors de la création d'une nouvelle classe en C++, quelles affirmations relèvent de bonnes pratiques ?

a) Les attributs représentent l'état de la classe et les méthodes représentent le comportement.
b) Les attributs et méthodes d'une classe doivent contribuer à une même responsabilité.

c) Les méthodes servent uniquement à accéder ou modifier les attributs (getters/setters).
d) Le choix entre private, protected ou public pour les attributs n'est pas important, il suffit juste d'adapter le code de la classe.

Q05 Une méthode doit être déclarée const lorsque :

a) Elle ne modifie pas l'état de l'objet sur lequel elle porte.
b) Elle retourne une valeur calculée qui est de type const.
c) Elle est appelée depuis une autre méthode const.
d) Elle ne modifie que des variables locales à la méthode.

Q06 À propos des fonctions helper (fonction portant sur un objet d'une classe mais non membre de la classe), quelles affirmations sont correctes ?

a) Une fonction helper fait partie de la déclaration de la classe au même titre que ses méthodes.
b) Une fonction helper doit être déclarée friend de la classe pour accéder à ses données.
c) Une fonction helper permet d'ajouter un comportement à une classe sans modifier sa définition.
d) Une fonction helper est adaptée pour des opérations ne relevant pas directement de la responsabilité de la classe.

Q07 À propos de l'héritage en programmation orientée objet, quelles affirmations sont correctes ?

a) L'héritage doit être utilisé dès qu'une classe utilise une autre classe.
b) L'héritage permet de factoriser des attributs et des méthodes communs à plusieurs classes.
c) L'héritage est pertinent lorsqu'une classe est un cas particulier d'une autre classe plus générale.
d) L'héritage garantit qu'une modification de la classe mère n'aura pas d'impact sur les classes dérivées.

Q08 Concernant les constructeurs lors de l'héritage en C++, quelles affirmations sont correctes ?

a) Le constructeur de la classe dérivée prend comme premier paramètre un objet de la classe mère pour pouvoir initialiser l'objet mère.
b) Le constructeur de la classe mère est appelé avant celui de la classe dérivée.
c) L'objet mère peut être créé dans le corps du constructeur de la classe dérivée.
d) La liste d'initialisation est l'unique moyen pour construire l'objet mère.

Q09 Quel est l'intérêt principal de la STL en C++ ?

a) La STL fournit des conteneurs et des algorithmes génériques permettant de manipuler des données de manière efficace et réutilisable
b) La STL permet d'exprimer des traitements de haut niveau sur les conteneurs de données sans nécessiter une implémentation spécifique de ces traitements
c) Le recours à la STL permet de gagner du temps de développement.
d) La STL couvre tous les besoins de manipulations de données et évite d'avoir à créer des classes dédiées.

Q10 À propos des itérateurs et des algorithmes de la STL, quelles affirmations sont correctes ?

a) Les itérateurs définissent une manière standard d'accéder aux éléments d'un conteneur.
b) Les itérateurs sont des indices numériques permettant de se déplacer dans les conteneurs.
c) Les algorithmes peuvent être spécialisés grâce à des fonctions ou des expressions lambda.
d) Les algorithmes sont liés à un conteneur spécifique et ne peuvent pas être réutilisés avec d'autres types de conteneurs.

Exercice 2 : Modélisation d'une réunion (10 points)

On souhaite modéliser un système simple de gestion de réunions. Une réunion est représentée par une classe `Meeting`. Chaque réunion est caractérisée par un titre (`std::string`), une date (`Datetime`) et une durée exprimée en minutes (`int`), un ensemble de participants (`Person`).

Les classes `Datetime` et `Person` sont fournies (voir déclaration ci-dessous).

datetime.h

```
class DateTime {
private:
  int year_;
  int month_;   // 1 à 12
  int day_;     // 1 à 31
  int hour_;    // 0 à 23
  int minute_;  // 0 à 59
public:
  DateTime(int year, int month, int day,
int hour, int minute);
  int year() const;
  int month() const;
  int day() const;
  int hour() const;
  int minute() const;

  // Comparaison chronologique
 bool operator<(const DateTime& other)
const;

  // Ajout d'un nombre de minutes
  DateTime add_minutes(int minutes)
const;
};
// Conversion en «YY/MM/DD HH:MM»
std::string to_string(const DateTime& d);
```

person.h

```
class Person {
private:
  std::string firstname_;
  std::string lastname_;
  std::string mail_;

public:
  Person(const std::string& firstname,
         const std::string& lastname,
         const std::string& mail);

  std::string firstname() const;
  std::string lastname() const;
  std::string mail() const;

  // Deux personnes sont identiques si
  // leurs adresses mails sont identiques
  bool operator==(const Person& other)
const;
};

// Conversion en «lastname firstname
(mail)»
std::string to_string(const Person& p);
```

Contraintes et hypothèses

- Le titre d'un objet `Meeting` ne doit pas être vide.
- La durée d'un `Meeting` doit être strictement positive.
- Une réunion peut être créée sans participant.

Q1 - Déclaration de la classe `Meeting`: Écrire la déclaration de la classe `Meeting`, en définissant les attributs nécessaires et les méthodes correspondantes.

Q2 - Constructeur et getters : Écrire le code du constructeur et des getters de `Meeting`.

Q3 - Nombre de participants : Écrire le code de la méthode qui renvoie le nombre de participants de type `Person`.

Q4 — Vérification de présence : Écrire le code de la méthode qui vérifie si une `Person` fait partie des participants d'un `Meeting`.

Q5 - Ajout d'un participant : Écrire le code de la méthode qui ajoute un participant à un `Meeting` uniquement s'il n'est pas déjà présent. La méthode indiquera si l'ajout a bien été effectué.

Q6 - Suppression d'un participant : Écrire le code de la méthode qui supprime un participant d'un `Meeting` uniquement s'il est présent. La méthode indiquera si la suppression a bien été effectuée.

Q7 - Fin d'un `Meeting` : Écrire le code de la méthode qui calcule le `Datetime` de la fin d'un `Meeting` à partir du `Datetime` et de la durée du `Meeting`.

Q8 – Chevauchement de 2 `Meeting` : Écrire le code de la méthode qui vérifie si le `Meeting` courant chevauche un autre `Meeting` passé en paramètre. Deux `Meeting` se chevauchent si leurs plages horaires ont une partie commune.

Q9 – Ajout de plusieurs participants : Écrire le code de la méthode qui ajoute plusieurs `Person` (contenus dans un tableau) à un `Meeting`. Une `Person` n'est ajoutée que si elle n'est pas déjà présente dans le `Meeting`. La méthode renverra le nombre de `Person` effectivement ajoutées.

Q10 - Tri des participants : Ecrire le code de la méthode qui trie par ordre alphabétique les `Person` d'un `Meeting` (`Lastname` en priorité, puis `Firstname` ensuite).

Exercice 3 : Analyse de code et détection d'erreur (5 points)

On considère le code C++ suivant qui définit deux classes : la classe `Document` représente un document générique et la classe `SignedDocument` un document qui peut être signé.

Trouver et expliquer les 5 erreurs présentes dans le code, puis proposer une correction pour chacune. Attention, les erreurs ne concernent pas la validation des données.

document.h

```
#include <string>

class Document {
private:
  std::string title_;
  int pagecount_;
public:
  Document(const std::string& t,
           int p);
  std::string title() const;
  void setTitle(const std::string& t);
  int pageCount() const;
  int addPage() const;
};
void rename(Document doc,
            const std::string& t);
```

document.cpp

```
#include "document.h"

Document::Document(const std::string& t, int p)
  : title_(t), pagecount_(p) {
}
std::string Document::title() const {
  return title_;
}
void Document::setTitle(const std::string& t) {
    title_ = t;
}
int Document::pageCount() const {
  return pagecount_;
}
int Document::addPage() const {
  return ++pagecount_;
}
void rename(Document doc, const std::string& t){
  doc.setTitle(t);
}
```

signed_document.h

```
#include "document.h"
#include <string>

class SignedDocument : public Document {
private:
  std::string signername_;
public:
  SignedDocument(const std::string& t,
                 int p,
                 const std::string& s);
  void sign(const std::string& s);
  void defaultSign() const;
  bool isSigned() const;
  std::string signerName () const;
  bool isImportant() const;

};
```

signed_document.cpp

```
#include "signed_document.h"

SignedDocument::SignedDocument(const  std::string&  t,
int p, const std::string& s)
  : title_(t), pagecount_(p), signername_(s) {
}
void SignedDocument::sign(const std::string& s) {
  signername_ = s;
}
void SignedDocument::defaultSign() const {
    if (!isSigned()) {
        sign("default");
    }
}
bool SignedDocument::isSigned() const {
    return !signername_.empty();
}
std::string SignedDocument::signerName() const {
    return signername_;
}
bool SignedDocument::isImportant() const {
    return isSigned() && pagecount_ > 10;
}
```